SAP-1 Mini Project Report

1. Overview

This project implements a simple SAP-1 style CPU in Verilog. The design is an 8-bit, single-bus, hardwired system with 16 bytes of memory. The CPU uses the following major components:

* Program counter (`PC`)
* Memory address register (`MAR`)
* Instruction register (`IR`)
* Register `A`
* Register `B`
* Arithmetic logic unit (`ALU`)
* Memory
* Single shared bus
* Hardwired control unit

The original assignment program is:

```text
Address  Data   Meaning
0        03     LDA 3
1        16     ADD 6
2        F0     HLT
3        04     Data value 4
6        05     Data value 5
```

Instruction encoding used in this implementation:

* `0x0_` = `LDA addr`
* `0x1_` = `ADD addr`
* `0x2_` = `SUB addr`
* `0x3_` = `STA addr`
* `0x4_` = `MUL addr`
* `0x5_` = `DIV addr`
* `0xE_` = `NOP`
* `0xF_` = `HLT`

This repository now also supports multiplication and division. The extended validation program used in the current testbench is:

```text
Address  Data   Meaning
0        09     LDA 9
1        16     ADD 6
2        47     MUL 7
3        58     DIV 8
4        F0     HLT
6        05     Data value 5
7        02     Data value 2
8        03     Data value 3
```

```
9        04     Data value 4
```

2. Module Breakdown

* `cpu.v`: connects all modules and exposes internal debug signals
* `control_unit.v`: generates timing steps and control signals
* `register.v`: reusable register block for MAR, IR, A, and B
* `program_counter.v`: holds the next instruction address
* `alu.v`: performs add, subtract, multiply, and divide on `A` and `B`
* `memory.v`: 16-byte memory array with asynchronous read and synchronous write
* `bus.v`: selects one data source onto the single 8-bit bus

3. Control Signals

The controller generates the following signals:

* `CO`: place `PC` on the bus
* `CE`: increment the program counter
* `MI`: load `MAR` from the bus
* `RO`: place memory data on the bus
* `RI`: write bus data into memory at `MAR`
* `II`: load `IR` from the bus
* `IO`: place the low nibble of `IR` on the bus
* `AI`: load register `A` from the bus
* `AO`: place register `A` on the bus
* `BI`: load register `B` from the bus
* `EO`: place ALU result on the bus
* `SU`: tell the ALU to subtract instead of add
* `MU`: tell the ALU to multiply
* `DU`: tell the ALU to divide
* `HLT`: halt the processor

4. Timing-Step Explanation

4.1 Fetch cycle for every instruction

T0

* `CO = 1`, `MI = 1`
* `PC` drives the bus.
* `MAR` loads the bus value.
* Result: the memory address register now points to the current instruction.

T1

* `RO = 1`, `II = 1`, `CE = 1`
* Memory places `mem[MAR]` on the bus.
* `IR` loads the instruction from the bus.
* `PC` increments to the next instruction address.
* Result: the instruction has been fetched and decoded fields are now available through `IR[7:4]` and `IR[3:0]`.

4.2 Instruction `LDA 3`

The instruction at address `0` is `03`, so `opcode = 0` and `operand = 3`.

T2

* `IO = 1`, `MI = 1`
* The operand field `IR[3:0] = 3` is driven onto the bus.
* `MAR` loads address `3`.
* Result: memory is prepared to read the operand from address `3`.

T3

* `RO = 1`, `AI = 1`
* Memory places `mem[3] = 04` on the bus.
* Register `A` loads the bus value.
* Result: `A = 04`.

After `T3`, the control unit resets the timing counter to `T0`, so the next instruction starts its own fetch cycle.

4.3 Instruction `ADD 6`

The instruction at address `1` is `16`, so `opcode = 1` and `operand = 6`.

T0

* `CO = 1`, `MI = 1`
* `PC = 1` drives the bus and `MAR` becomes `1`.

T1

* `RO = 1`, `II = 1`, `CE = 1`
* Memory places `mem[1] = 16` on the bus.
* `IR` becomes `16`.
* `PC` increments from `1` to `2`.

T2

* `IO = 1`, `MI = 1`
* The low nibble of `IR` drives `6` onto the bus.
* `MAR` becomes `6`.
* Result: the operand address is selected.

T3

* `RO = 1`, `BI = 1`
* Memory places `mem[6] = 05` on the bus.
* Register `B` loads `05`.
* Result: `B = 05`.

T4

* `EO = 1`, `AI = 1`, `SU = 0`
* The ALU computes `A + B = 04 + 05 = 09`.
* The ALU result drives the bus.
* Register `A` loads the ALU result.
* Result: `A = 09`.

After `T4`, the timing counter resets to `T0`.

4.4 Instruction `HLT`

The instruction at address `2` is `F0`, so `opcode = F`.

T0

* `CO = 1`, `MI = 1`
* `PC = 2` is copied into `MAR`.

T1

* `RO = 1`, `II = 1`, `CE = 1`
* Memory places `mem[2] = F0` on the bus.
* `IR` loads `F0`.
* `PC` increments to `3`.

T2

* `HLT = 1`
* The control unit latches the halted state.
* The timing counter resets and the CPU stops advancing.

4.5 Instruction `MUL 7`

In the extended validation program, the instruction at address `2` is `47`, so `opcode = 4` and `operand = 7`.

T2

* `IO = 1`, `MI = 1`
* The operand field `IR[3:0] = 7` is driven onto the bus.
* `MAR` loads address `7`.

T3

* `RO = 1`, `BI = 1`
* Memory places `mem[7] = 02` on the bus.
* Register `B` loads `02`.

T4

* `MU = 1`, `EO = 1`, `AI = 1`
* The ALU computes `A * B = 09 * 02 = 12` hexadecimal, or `18` decimal.
* The ALU result drives the bus.
* Register `A` loads the product.

* Result: `A = 12`.

4.6 Instruction `DIV 8`

In the extended validation program, the instruction at address `3` is `58`, so `opcode = 5` and `operand = 8`.

T2

* `IO = 1`, `MI = 1`
* The operand field `IR[3:0] = 8` is driven onto the bus.
* `MAR` loads address `8`.

T3

* `RO = 1`, `BI = 1`
* Memory places `mem[8] = 03` on the bus.
* Register `B` loads `03`.

T4

* `DU = 1`, `EO = 1`, `AI = 1`
* The ALU computes `A / B = 12 / 03 = 06` hexadecimal, or `6` decimal.
* The ALU result drives the bus.
* Register `A` loads the quotient.
* Result: `A = 06`.

5. Final Result

At the end of the program:

* Assignment reference program:
  - `LDA 3` loads `4` into `A`
  - `ADD 6` updates `A` to `9`
* Extended validation program:
  - `LDA 9` gives `A = 4`
  - `ADD 6` gives `A = 9`
  - `MUL 7` gives `A = 18`
  - `DIV 8` gives `A = 6`

So the updated machine correctly performs:

```text
LDA 9   => A = mem[9] = 4
ADD 6   => A = A + mem[6] = 4 + 5 = 9
MUL 7   => A = A * mem[7] = 9 * 2 = 18
DIV 8   => A = A / mem[8] = 18 / 3 = 6
```

For division by zero, this implementation returns `00` to keep the ALU behavior defined in simulation.

6. Waveform Notes

For waveform capture in Vivado, add all files in `src/` and `tb/cpu_tb.v`, run behavioral simulation, and include the following signals in the waveform window:

* `clk`
* `reset`
* `t_state`
* `pc_value`
* `mar_value`
* `ir_value`
* `bus_value`
* `a_value`
* `b_value`
* `mem_value`
* `alu_value`
* `co ce mi ro ri ii io ai ao bi eo su mu du hlt`
* `halted`

The key waveform moments to capture are:

* Fetch of `LDA 9`
* Execution of `LDA 9` where `A` becomes `04`
* Execution of `ADD 6` where `B` becomes `05`
* ALU multiply step where `A` becomes `12`
* ALU divide step where `A` becomes `06`
* Halt step where `HLT` goes high and `halted` becomes `1`